

云原生批量计算的集大成者 Volcano Job

CONTENTS

01 Volcano Job介绍

02 Volcano Job主要特性和应用场景

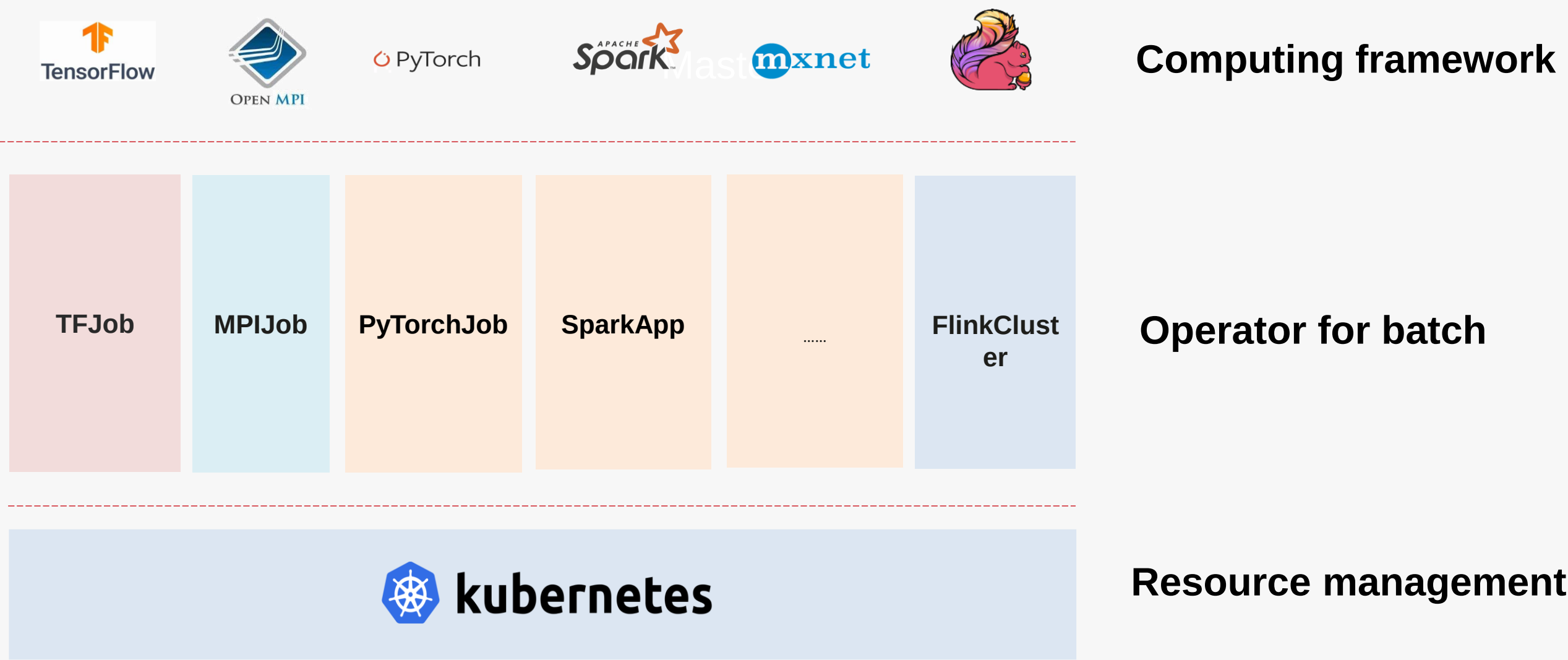


01

Volcano Job介绍



Batch Job 现状和挑战



现状:

- Kubernetes Job API 语义不足，无法描述复杂 Batch 负载
- 为了满足批量计算多样性的需求，各个领域各自开发和维护了自己的 Operator

挑战:

- 运维复杂、成本高，作业管理缺少统一的体验
- 缺少的完善的作业生命周期管理策略
- 缺少作业依赖、任务依赖等特性
- 扩展性不足、缺乏和调度器的协同

Volcano Job 介绍



Computing framework

Operator for batch

Resource management

Volcano Job 优势:

- 通用作业管理API, 统一支持多样性的计算框架, 如 mpi, pytorch, tensorflow, mxnet 等
- 统一体验、降低运维复杂性
- 精细化的作业生命周期管理
- 灵活的扩展机制, 满足定制化场景需求
 - ✓ Env
 - ✓ Svc
 - ✓ Ssh
 - ✓ Tensorflow
 - ✓ ...
- 增强作业控制器和调度器的协同
- 提供作业依赖, 任务依赖等关键特性

02

主要特性和应用场景



Volcano Job API

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: test-job
spec:
  minAvailable: 3
  schedulerName: volcano
  policies:
    - event: PodEvicted
      action: RestartJob
  plugins:
    ssh: []
    env: []
    svc: []
  maxRetry: 5
  tasks:
    - replicas: 6
      name: "default-nginx"
      template:
        metadata:
          name: web
        spec:
          containers:
            - image: nginx
              imagePullPolicy: IfNotPresent
              name: nginx
              resources:
                requests:
                  cpu: "1"
```

schedulerName 指定使用的调度器的名字

minAvailable 代表只有当minAvailable数量个Pod资源都满足作业才会一起调度起来

policies: 配置作业生命周期管理策略

plugins: 扩展Job API, 定义定制化需求

maxRetry: 指定Job失败前最大重试次数

tasks.name 指定Job中Task的类型

tasks.replicas 指定Task的副本数



Multiple Pod template

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: tensorflow-dist-mnist
  labels:
    "volcano.sh/job-type": "tensorflow"
spec:
  # minimum number of pods need to be started
  minAvailable: 6
  schedulerName: volcano
  tasks:
    - replicas: 2
      name: ps
      template:
        spec:
          containers:
            .....
            image: volcanosh/dist-mnist-tf-example:0.0.1
    - replicas: 4
      name: worker
      template:
        spec:
          containers:
            .....
            image: volcanosh/dist-mnist-tf-example:0.0.1
```

场景:

大多数的批量计算的负载均包含多种不同的任务类型，如 TensorFlow (ps/worker), MPI(master/worker). 同时不同的任务类型使用的镜像，需要的资源各不相同，比如Tensorflow的ps主要使用CPU资源，而worker倾向于使用GPU资源进行加速。

tasks.name 指定Job中Task的类型

Tasks.replicas 指定Task的副本数



作业优先级

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: tensorflow-dist-mnist
  labels:
    "volcano.sh/job-type": "tensorflow"
spec:
  # minimum number of pods need to be started
  minAvailable: 3
  schedulerName: volcano
  priorityClassName: high
  tasks:
    - replicas: 2
      name: ps
      template:
        spec:
          containers:
            .....
            image: volcanosh/dist-mnist-tf-example:0.0.1
    - replicas: 4
      name: worker
      template:
        spec:
          containers:
            .....
            image: volcanosh/dist-mnist-tf-example:0.0.1
```

场景:

生产环境环境中，通常会存在一些紧急的，或者是重要的应用负载，这些负载的SLA需要优先保障

Job优先级:

优先级代表该作业的重要性，高优先级作业优先获得集群资源，在资源不足时，允许高优先级作业抢占低优先级作业来获取资源。



任务优先级

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: pytorch-job
spec:
  minAvailable: 3
  schedulerName: volcano
  plugins:
    pytorch: ["--master=master", "--worker=worker", "--port=23456"]
  tasks:
    - replicas: 1
      name: master
      template:
        spec:
          priorityClassName: high
          containers:
            - image: gcr.io/kubeflow-ci/pytorch-dist-sendrecv-test:1.0
              name: master
    - replicas: 2
      name: worker
      template:
        spec:
          containers:
            - image: gcr.io/kubeflow-ci/pytorch-dist-sendrecv-test:1.0
              imagePullPolicy: IfNotPresent
              name: worker
              workingDir: /home
```

场景:

在一个作业内部，存在不同类型的任务，不同类型的任务的重要程度有可能是不同的，比如Pytorch作业中国的master和worker，Spark作业中的driver和executor。

任务优先级:

Volcano允许用户针对不同任务类型配置不同的优先级，保证高优先级任务对资源的优先获取。



作业队列

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: test-job
spec:
  queue: test-queue
tasks:
  - replicas: 1
    name: "test"
    template:
      spec:
        containers:
          - image: alpine
            command: ["/bin/sh", "-c", "sleep 1000"]
            imagePullPolicy: IfNotPresent
            name: test-job
            resources:
              requests:
                cpu: "0.1"
```

```
apiVersion: scheduling.volcano.sh/v1beta1
kind: Queue
metadata:
  name: test-queue
spec:
  weight: 1
```

场景:

Queue 是批量计算系统非常重要的概念，通过Queue 可以实现多租户的资源共享，同时也提供了一定程度的资源隔离。Volcano 允许指定Job提交到指定 Queue



任务索引

```
apiVersion: batch.volcano.sh/v1alpha1
```

```
kind: Job
```

```
metadata:
```

```
  name: "test-job"
```

```
spec:
```

```
  plugins:
```

```
  env: []
```

```
  minAvailable: 1
```

```
  schedulerName: volcano
```

```
  tasks:
```

```
    - replicas: 3
```

```
      name: nginx
```

```
      template:
```

```
        spec:
```

```
          containers:
```

```
            - image: alpine
```

```
              command: ["/bin/sh", "-c", "echo my index is ${VC_TASK_INDEX}"]
```

```
              imagePullPolicy: IfNotPresent
```

```
              name: test-job
```

```
              resources:
```

```
                requests:
```

```
                  cpu: 1
```

```
              restartPolicy: OnFailure
```

场景:

在一些场景下，运行并行的作业，你需要不同的Task处理不同的数据分片，此时运行的Pod需要知道自己在Job中的索引

Task索引

Volcano 允许Index 以环境变量VC_TASK_INDEX的方式暴露给Pod，每个Pod通过Index分析该Pod需要处理的数据分片，这样用户就无需构建一个单独的系统来告诉Pod需要处理那个数据分片。



作业生命周期管理

Job transactions between phases

From\To	Pending	Aborted	Running	Completed	Terminated
Pending	*	*	*		
Aborted	*	*			
Running		*	*	*	*
Completed				*	
Terminated					*

作业状态：Job 包含了多个不同的状态, 表格展示了作业不同状态的转换

作业在系统中创建完成后，作业中Pod会有不同的事件，比如MPI Job的Task failure. Job 生命周期策略用于针对用户的配置，处理这些事件。

Error handing

Pod Event list	Action list
PodFailed	AbortJob
PodEvicted	RestartJob
Unknown	TerminateJob
OutOfSync	CompleteJob
CommandIssued	ResumeJob
TaskCompleted	SyncJob

作业策略

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: tensorflow-dist-mnist
  labels:
    "volcano.sh/job-type": "tensorflow"
spec:
  minAvailable: 3
  schedulerName: volcano
  policies:
    - event: PodEvicted
      action: RestartJob
  tasks:
    - replicas: 2
      name: ps
      template:
        spec:
          containers:
            .....
            image: volcanosh/dist-mnist-tf-example:0.0.1
    - replicas: 4
      name: worker
      policies:
        - event: TaskCompleted
          action: CompleteJob
      template:
        .....
```

场景:

同一个作业内，不同类型的任务需要不同的策略，
Volcano 允许给指定类型任务配置策略。



任务依赖

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: example-job
spec:
  .....
  tasks:
    - replicas: 2
      name: task1
      template:
        .....
    - replicas: 1
      name: task2
      template:
        .....
    - replicas: 2
      name: task3
      dependsOn:
        name: task1, task2
        iteration: any
      template:
        .....
```

场景:

对于一部分的批量作业，作业内部的任务之间存在依赖关系，以MPI 作业为例，worker pods 需要先启动配置ssh服务，然后启动master pod 连接worker pods，否则master pod有可能无法和worker pods成功建立连接而导致失败，造成不必要的资源浪费。

dependsOn 用于指定当前Task所依赖的Task列表

作业存活时间

```
apiVersion: batch.volcano.sh/v1alpha1
kind: job
metadata:
  generateName: test-job
spec:
  minAvailable: 1
  schedulerName: volcano
  ttlSecondsAfterFinished: 600
  policies:
    - event: PodEvicted
      action: RestartJob
  tasks:
    - replicas: 1
      name: sleeper
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: sleeper
              image: debian:buster
              command:
                - /bin/bash
                - -c
                - |
                  for i in {0..5}; do
                    echo "sleeping"
                    sleep 1
                  done
```

场景:

默认情况下, Job运行结束后会继续保留在系统中, 在大规模集群中, 大量的完成作业会造成ETCD的压力, 影响集群性能。

ttlSecondsAfterFinished 允许用户配置一个TTL, Job Controller 会自动进行资源回收。



作业 sla-waiting-time

```
apiVersion: batch.volcano.sh/v1alpha1
```

```
kind: Job
```

```
metadata:
```

```
  annotations:
```

```
    sla-waiting-time: 1h
```

```
spec:
```

```
  # minimum number of pods need to be started
```

```
  minAvailable: 6
```

```
  schedulerName: volcano
```

```
  tasks:
```

```
    - replicas: 2
```

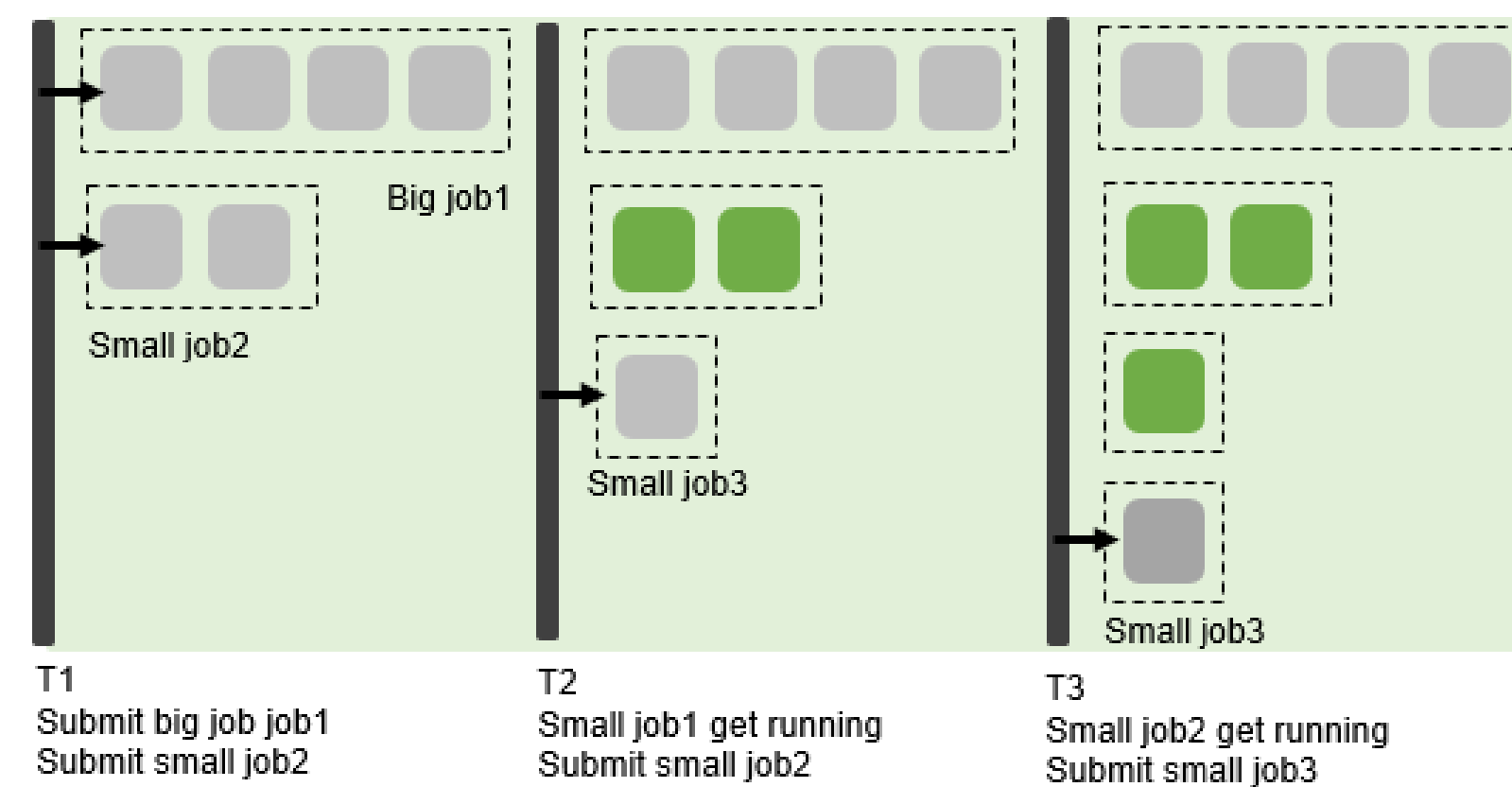
```
      name: ps
```

场景:

集群中大作业，小作业并存的场景，会产生资源竞争，资源如果持续紧张，有可能会发生大作业/小作业饿死，

Volcano允许用户为大作业配置sla-waiting-time，一旦条件满足，系统会为作业进行资源预留，防止作业饿死。

Job Stream



任务拓扑关系

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: tensorflow-dist-mnist
  labels:
    "volcano.sh/job-type": "tensorflow"
  annotations:
    volcano.sh/task-topology-affinity: "ps,worker;ps,evaluator"
    volcano.sh/task-topology-anti-affinity: "ps;worker,chief;chief,evaluator"
    volcano.sh/task-topology-task-order: "ps,worker,chief,evaluator"
spec:
  minAvailable: 6
  schedulerName: volcano
  tasks:
    - replicas: 2
      name: ps
      template:
        spec:
          containers:
            .....
            image: volcanosh/dist-mnist-tf-example:0.0.1
    - replicas: 4
      name: worker
      template:
        spec:
          containers:
            .....
            image: volcanosh/dist-mnist-tf-example:0.0.1
```

场景:

在一些大数据、AI场景下，Job内Task之间的数据传输，尤其是跨节点的数据拷贝带来了很高的性能损耗，甚至成为整个作业性能的瓶颈。

Volcano允许用户配置Task的拓扑关系，调度器依据拓扑关系实现最优调度，减少节点间数据的拷贝，提升作业整体性能。



Job cooldown-time

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: pytorch-job
annotations:
  volcano.sh/cooldown-time: "600s"
spec:
  minAvailable: 3
  schedulerName: volcano
  plugins:
    pytorch: ["--master=master", "--worker=worker", "--port=23456"]
  tasks:
    - replicas: 1
      name: master
      template:
        spec:
          containers:
            - image: gcr.io/kubeflow-ci/pytorch-dist-sendrecv-test:1.0
              name: master
    - replicas: 9
      name: worker
      template:
        spec:
          containers:
            - image: gcr.io/kubeflow-ci/pytorch-dist-sendrecv-test:1.0
              imagePullPolicy: IfNotPresent
              name: worker
              workingDir: /home
```

场景:

在弹性训练、推理的场景，弹性的Pod 容易被无限制的抢占，可能刚获得资源刚启动，就被其他作业抢占，导致Pod状态稳定性不足。

volcano.sh/cooldown-time 允许用户配置一个时间值，在这个时间值代表该Pod的冷静期，在冷静期内，Pod 将不会被驱逐。



作业插件

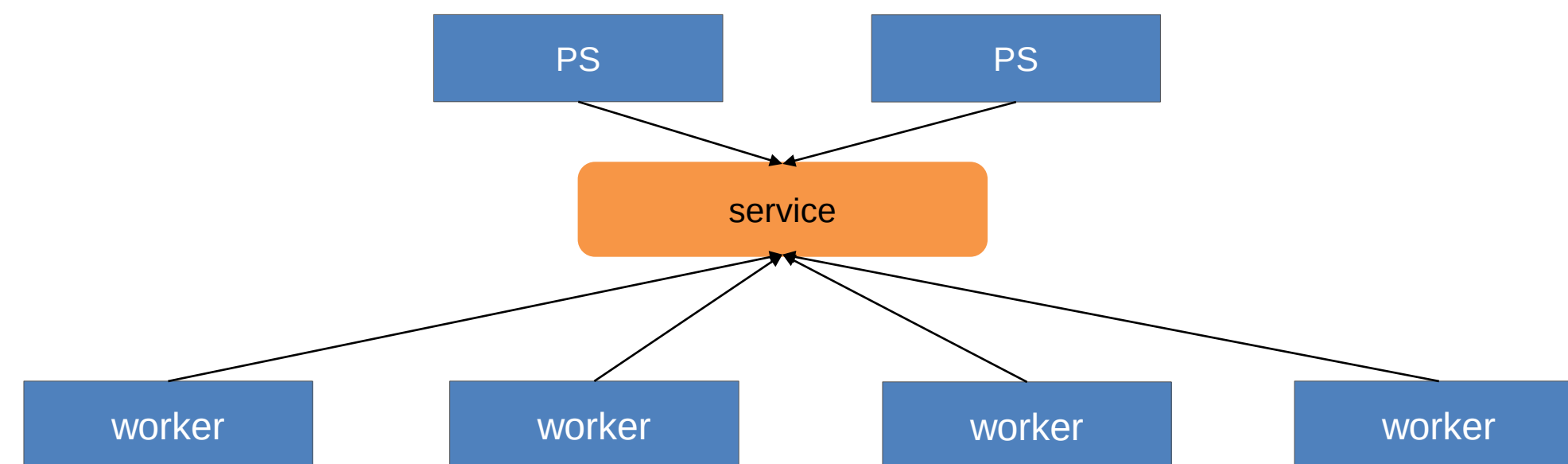
```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: mpi-job
spec:
  minAvailable: 2
  schedulerName: volcano
  policies:
    - event: PodEvicted
      action: RestartJob
  plugins:
    ssh: []
    env: []
    svc: []
  tasks:
    - replicas: 1
      name: mpimaster
      template:
        spec:
          containers:
            image: mpi-image
            name: mpimaster
    - replicas: 2
      name: mpiworker
      template:
        spec:
          containers:
            image: mpi-image
            name: mpiworker
```

场景:

许多AI 计算框架都需要设置环境变量、pod间的互访以及ssh免密登录等，Job插件提供一种扩展方式，允许定义一些定制化的需求。Volcano默认自带了ssh, env, svc, mpi, pytorch job插件。简化用户配置，让用户更关注的业务。



任务之间的通信



```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: tensorflow-dist-mnist
  labels:
    "volcano.sh/job-type": "tensorflow"
spec:
  minAvailable: 3
  schedulerName: volcano
  plugins:
    svc: []
    ssh: []
    env: []
  tasks:
    - replicas: 2
      name: ps
      .....
```

场景:

在Tensorflow PS-Worker架构下, The Parameter server pods 存储模型参数, worker pods 进行梯度计算. 在每次迭代计算时 workers 从Parameter server pods 获取参数进行计算, 然后返回计算结果. Parameter servers 聚合计算结果, 更新参数, 最后广播参数到各个worker。

svc plugin: 用于创建headless service解决ps和worker之间的通信问题

本课回顾

- 通用 Job API

- Job multiple pod template
- Job priority, task priority
- Job queue
- Job sla-waiting-time
- Task topology
- Job cooldowntime

- 作业生命周期管理

- Job State
- Job Policy
- Task Policy
- Job maxRetry
- Job TTL

- 作业扩展

- Job dependency
- Task dependency
- Job plugin
- Task communication
- Task index

Join Volcano



Website: <https://volcano.sh/en/>



Github: <https://github.com/volcano-sh/volcano>



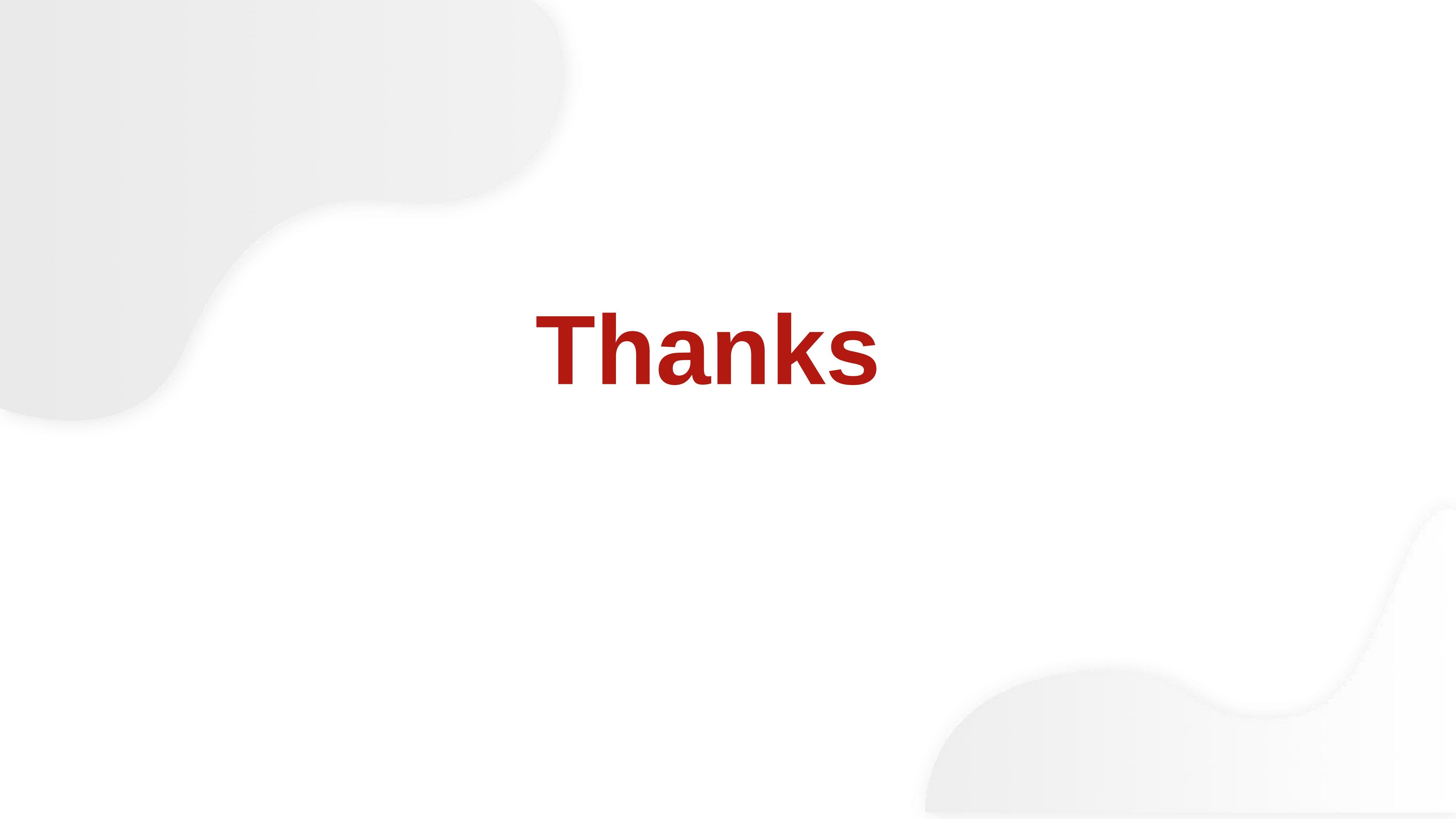
Slack Channel: <https://volcano-sh.slack.com/>



Follow Us



Join Group



Thanks